

AI Operations (AIOps)

Module 1 Assignment — Experiment Management & Reproducibility

TA Edition: includes model solutions, rubrics, and partial-credit guidance. Remove the “Model Solution” sections to distribute a clean student copy.

Total Marks	Weight	Submission
50	10% of course grade	Git repository link + a 1-page PDF write-up

Question 1 — Conceptual: Technical Debt Diagnosis (10 marks)

A campus food-delivery startup has a recommendation model in production. Over the last two months, engineers report: (a) changing the “estimated delivery time” feature’s rounding logic unexpectedly hurt recommendations for a completely unrelated “favorite restaurants” feature; (b) a marketing dashboard team has been silently reading the model’s raw output table for a campaign and nobody on the ML team knew; (c) the training pipeline is a chain of 14 undocumented shell scripts with no orchestration tool.

1. Identify which of the six hidden-technical-debt categories (from Lecture 1) best explains each of (a), (b), and (c). (6 marks)
2. For ONE of the three, propose a specific mitigation using a tool or practice introduced in Module 1 or previewed for later modules. (4 marks)

Model Solution

Scenario	Debt Category	Justification
(a) unrelated feature affected	Entanglement (CACE)	No code path connects the two features; the coupling is purely statistical — textbook CACE.
(b) marketing silently reads output	Undeclared Consumers	A downstream team depends on the model’s output without the ML team’s knowledge, blocking safe changes.
(c) 14 undocumented shell scripts	Glue Code / Pipeline Jungle	A tangle of stitched-together scripts with no orchestration layer, exactly the pattern described in the course content.

Mitigation examples (any one, well-justified, earns full 4 marks): For (a), instrument systematic experiment tracking (MLflow) so any parameter/feature change is logged and its downstream metric impact is comparable run-to-run, making entanglement effects visible rather than silent. For (b), formally register consumers/contracts on the output table, or migrate to a governed Feature Store (Module 2 preview) so consumption is declared. For (c), replace the shell-script chain with an orchestrated DAG (Airflow/Kubeflow, Module 2 preview).

Rubric: 2 marks per correct category ID (1 mark if category is close but justification is weak); 4 marks for a mitigation that is specific and correctly tied to a Module 1 tool/concept (2 marks if generic, e.g., “add more testing” with no tool named).

Question 2 — Applied: MLflow Experiment Comparison (15 marks)

Using the training script provided in the Lecture 3 exercises, run at least SIX experiments varying at least two hyperparameters (e.g., learning rate and batch size, or architecture), after changing the predictor from RandomForest to an MLP. Also, change the dataset from IRIS to MNIST.

Submit:

1. A screenshot of the MLflow run-comparison table for all six runs. (4 marks)
2. A short written analysis (150–250 words) identifying: the best-performing run and why; any evidence of overfitting from the train_loss vs. val_accuracy trend; and which hyperparameter appears to have the larger effect on performance. (7 marks)
3. The exact mlflow.log_param / mlflow.log_metric code you added to the starter script. (4 marks)

Model Solution / Grading Guidance

There is no single “correct” numeric answer since results depend on the student's actual runs; the TA grades the process and reasoning, not a specific accuracy figure. Use this checklist:

Check	Marks
≥ 6 runs present, ≥ 2 hyperparameters varied, visible in one screenshot	4
Correctly identifies best run using the VALIDATION metric (not training loss)	2
Correctly reasons about overfitting (train_loss keeps falling while val_accuracy plateaus/drops)	3
Reasonable claim about which hyperparameter matters more, supported by the data shown	2
Code snippet includes log_param, log_metric (with step=), and at least one log_artifact / log_model call	4

Sample reference code for grading comparison:

```
with mlflow.start_run(run_name=f"lr{lr}-bs{bs}"):
    mlflow.log_param("lr", lr)
    mlflow.log_param("batch_size", bs)
    for epoch in range(epochs):
        mlflow.log_metric("train_loss", train_loss, step=epoch)
        mlflow.log_metric("val_accuracy", val_acc, step=epoch)
    mlflow.pytorch.log_model(model, "model")
```

Common deduction: students who report only the FINAL epoch's metrics rather than logging per-epoch (step=epoch) lose 1 mark under the code-quality line, since this defeats the purpose of trend analysis in part 2.

Question 3 — Applied: DVC Data Versioning & Rollback (10 marks)

Starting from the provided dataset (data.zip used in the class), let's create a CSV of the file names, which will have 1800 rows + a header line.

1. Initialize DVC in a Git repo and push (let's call it) v1 to a remote (only SSH remote or any S3 bucket are acceptable). (2 marks)
2. Simulate a data update: add new rows from new_labels.zip to create v2 (now, the row count should 2801), re-run dvc add, and commit v2 with a clear commit message. (3 marks)
3. Demonstrate rolling back to v1 using git checkout + dvc checkout, and include terminal output (screenshot or copy-pasted) proving the row count matches v1 exactly. (5 marks)

Model Solution

```
dvc init
dvc remote add -d localstore ../dvcstore
# create train_v1.csv from data.zip
dvc add data/train_v1.csv
git add data/train_v1.csv.dvc .gitignore
git commit -m "v1 dataset: baseline 1800 rows"
dvc push
# simulate update
# append to train_v1.csv from new_labels.zip
dvc add data/train_v1.csv
git add data/train_v1.csv.dvc
git commit -m "v2 dataset: added 1000 new rows"
dvc push
# rollback proof
git log --oneline      # identify v1 commit hash
git checkout <v1-commit-hash> -- data/train_v1.csv.dvc
dvc checkout
wc -l data/train_v1.csv  # should show 2800 (+1 header) rows, matching v1
```

Rubric: 2 marks for correct init/remote/push; 3 marks for a correctly committed v2 with clear message; 5 marks for rollback proof — award full marks only if the row count evidence is actually included, not just claimed. Common error: students git checkout the whole commit rather than the .dvc file, which also reverts their code — acceptable but note in feedback that isolating the .dvc file is the more surgical technique.

Question 4 — Capstone: End-to-End Reproducibility Drill (15 marks)

Consecutive roll numbers will become partners for this experiment. Working with an assigned partner, complete the full reproducibility protocol:

1. Partner A trains a model, logs the run in MLflow (params, metrics, seed, git_commit tag, artifact), versions the dataset with DVC, and commits code + .dvc file in the same Git commit. Registers the model and transitions it to “Staging.” (6 marks)
2. Partner A shares the repository with Partner B (no other communication about environment/data allowed).
3. Partner B reproduces the result using only: git clone, git checkout <commit>, dvc checkout, mamba env create -f environment.yml, and reruns the script. (6 marks) [mamba or conda or virtualenv]
4. Partner B logs a note in the MLflow run documenting whether the metric matched (within a stated tolerance) or explaining any discrepancy. (3 marks)

Model Solution / Grading Guidance

This question has no single fixed answer — it is assessed as a verifiable process. TAs should ask each pair to demonstrate the reproduction live or provide a short screen recording.

Check	Marks
Run logged with params, metrics, seed, git_commit tag, and an artifact/model	3
.dvc file and training code committed together in the SAME Git commit	3
Model registered and transitioned to Staging with visible stage history	2
reproduction shows only the permitted commands were used (no manual data/env sharing)	3
Reproduced metric matches within a stated, reasonable tolerance OR a plausible discrepancy is documented	2
Confirmation note is actually logged inside the MLflow run (not just emailed/reported separately)	2

Acceptable tolerance guidance for the TA: floating-point/hardware non-determinism differences under ~1% absolute accuracy, or under 0.02 in loss, should be treated as a successful reproduction if seeds and library versions were correctly pinned. Larger discrepancies should prompt the TA to check whether the environment.yml was actually pinned (--no-builds) and whether the seed was genuinely set in all libraries used (Python random, NumPy, and the DL framework).

Note to TA: this question is the strongest single signal of whether a student has internalized the module's central idea (versioning code+data+model+env together), and should be weighted accordingly in any borderline grade decisions.

Overall Rubric Summary

Question	Marks	Skill Assessed
Q1	10	Conceptual understanding of ML technical debt
Q2	15	Practical MLflow experiment tracking & analysis
Q3	10	Practical DVC data versioning & rollback
Q4	15	Integrated end-to-end reproducibility (capstone)
Total	50	